# Our Primary Tool: The Python Language

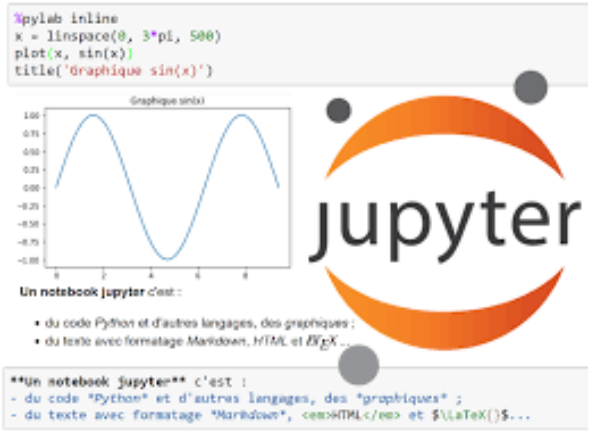
Why Python?

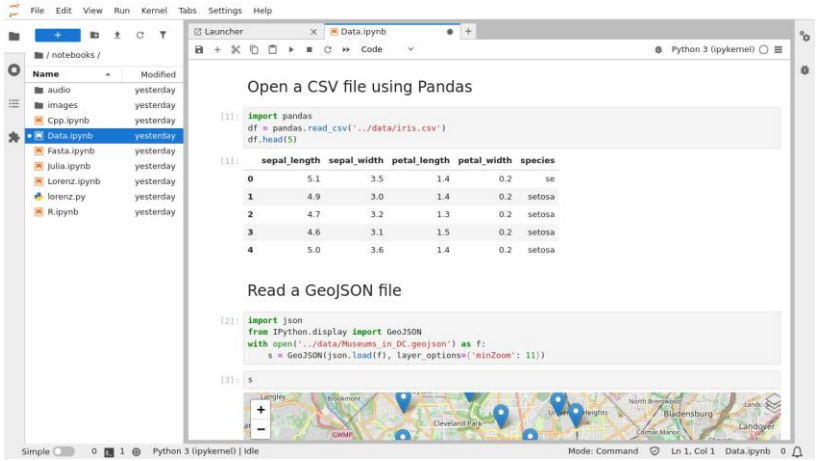
- **Readable and Simple**: Its syntax is clean and close to English, making it easy to learn.
- **Powerful Libraries**: It has a massive ecosystem of free, specialized toolkits for science (`NumPy, Pandas, Matplotlib`).
- **Versatile**: Used for everything from web development (Instagram, YouTube) to data science and AI. The skills you learn here are widely applicable.



# Our Lab Environment: Jupyter Notebook

- A Jupyter Notebook is an interactive web-based application for writing and running code.
- It allows you to create and share documents that contain:
  - Live code
  - Equations
  - Visualizations
  - Explanatory text
- It's the perfect digital lab notebook for this course.





# Getting Started with Jupyter Notebooks: Basic Operations

Working with Cells, Notebooks, and Presentations

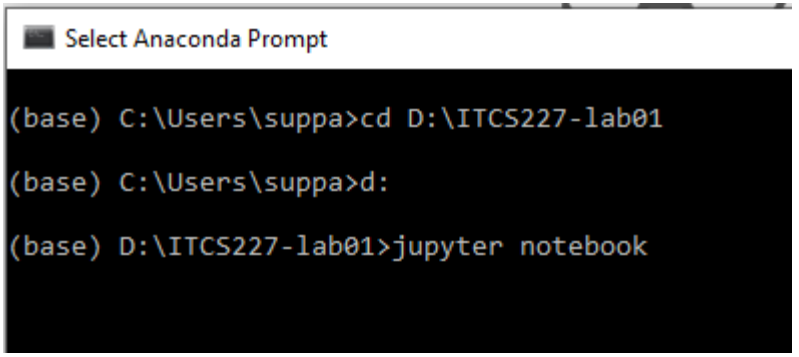
# Accessing Jupyter for this Course

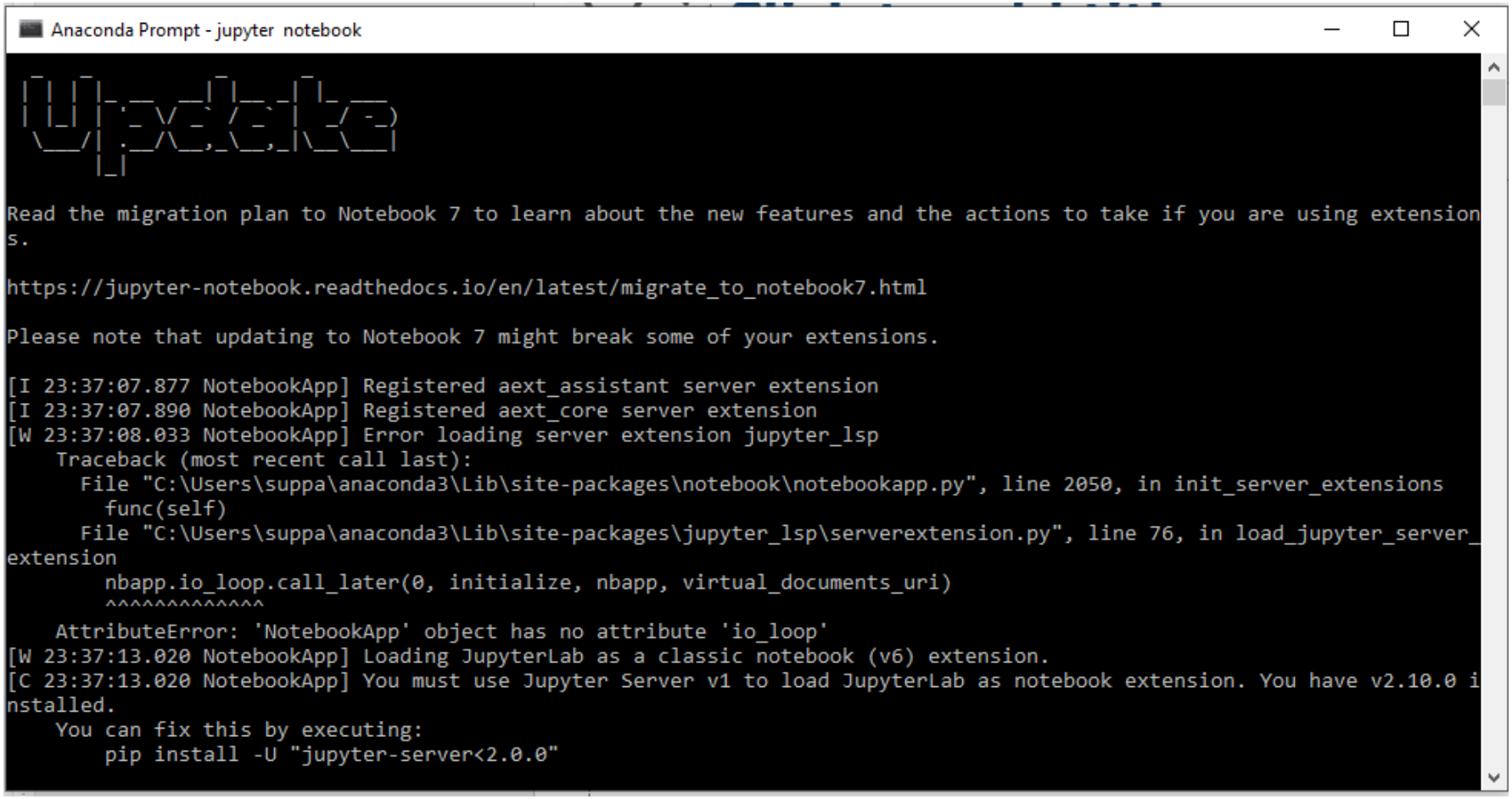
- Each lab computer should have Anaconda installed. If not, follow this tutorial: https://www.geeksforgeeks.org/install-jupyter-notebook-in-windows/

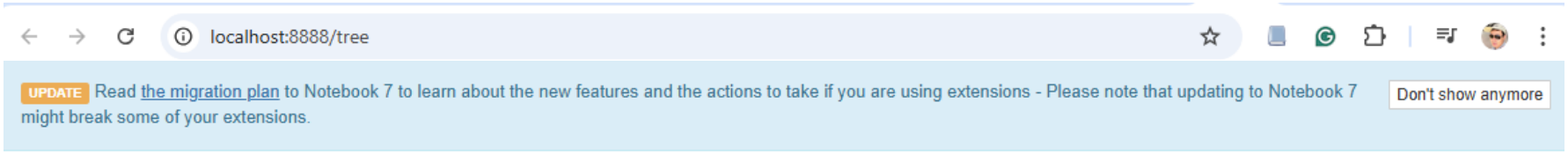
- Run "Anaconda Prompt"

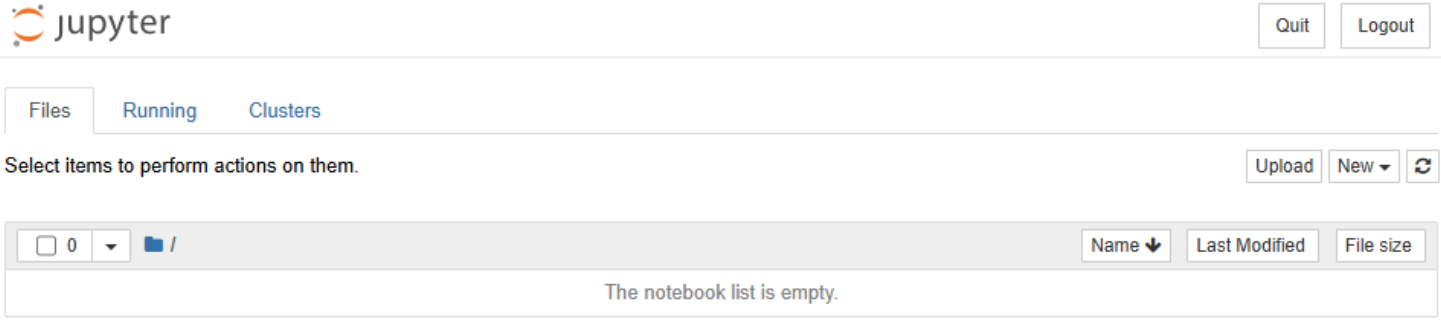
- Navigate to your working folder, e.g., "D:\ITGE162-lab01"
  - Use "cd <dir_path>"

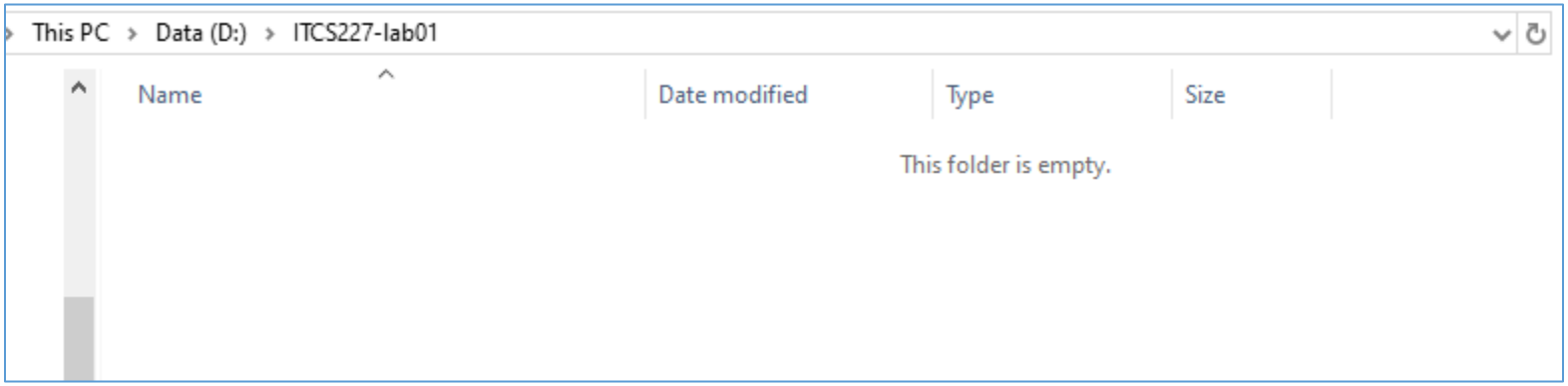
- Run "jupyter notebook"

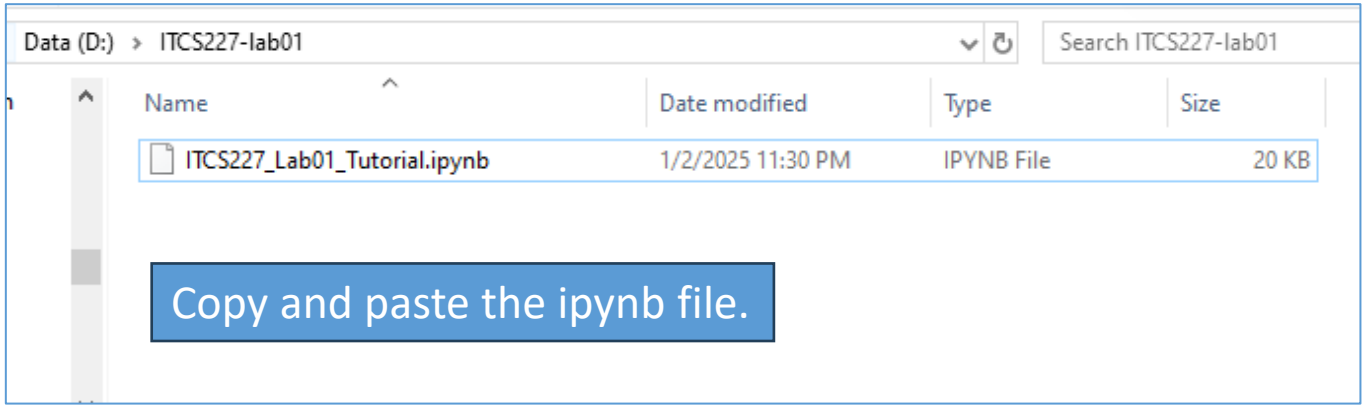


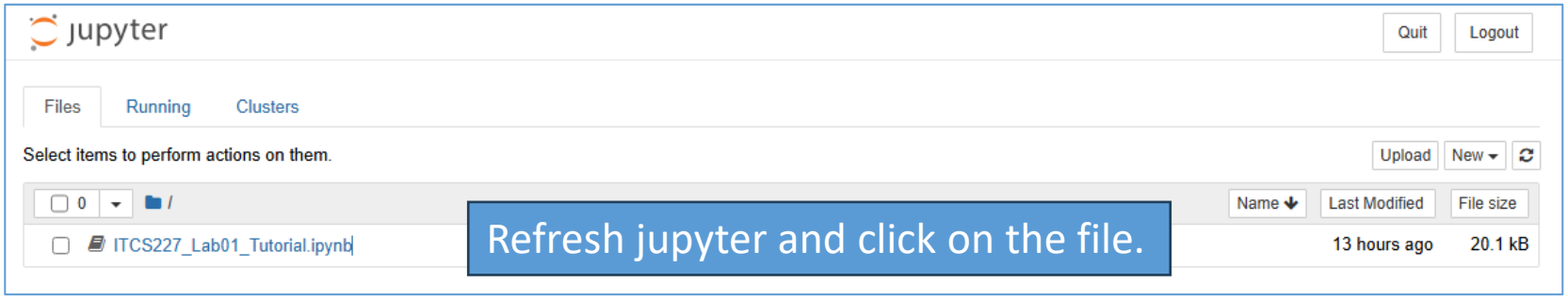

Anaconda Prompt - jupyter notebook
Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extension
s.
https://jupyter-notebook.readthedocs.io/en/latest/migrate_to_notebook7.html
Please note that updating to Notebook 7 might break some of your extensions.
[I 23:37:07.877 NotebookApp] Registered aext_assistant server extension
[I 23:37:07.890 NotebookApp] Registered aext_core server extension
[W 23:37:08.033 NotebookApp] Error loading server extension jupyter_lsp
Traceback (most recent call last):
File "C:\Users\suppa\anaconda3\Lib\site-packages\notebook\notebookapp.py", line 2050, in init_server_extensions
func(self)
File "C:\Users\suppa\anaconda3\Lib\site-packages\jupyter_lsp\serverextension.py", line 76, in load_jupyter_server_
extension
nbapp.io_loop.call_later(0, initialize, nbapp, virtual_documents_uri)
^^^^^^^^^^^^^
AttributeError: 'NotebookApp' object has no attribute 'io_loop'
[W 23:37:13.020 NotebookApp] Loading JupyterLab as a classic notebook (v6) extension.
[C 23:37:13.020 NotebookApp] You must use Jupyter Server v1 to load JupyterLab as notebook extension. You have v2.10.0 i
nstalled.
You can fix this by executing:
pip install -U "jupyter-server<2.0.0"

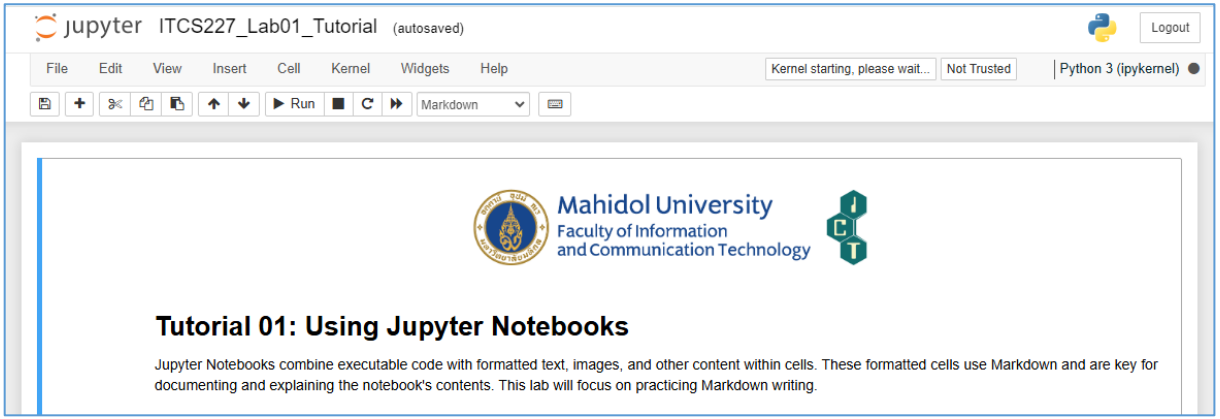

localhost:8888/tree
UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions.
Don't show anymore


jupyter
Quit
Logout
Files
Running
Clusters
Select items to perform actions on them.
Upload
New
0
/
Name
Last Modified
File size
The notebook list is empty.


This PC
Data (D:)
ITCS227-lab01
Name
Date modified
Type
Size
This folder is empty.

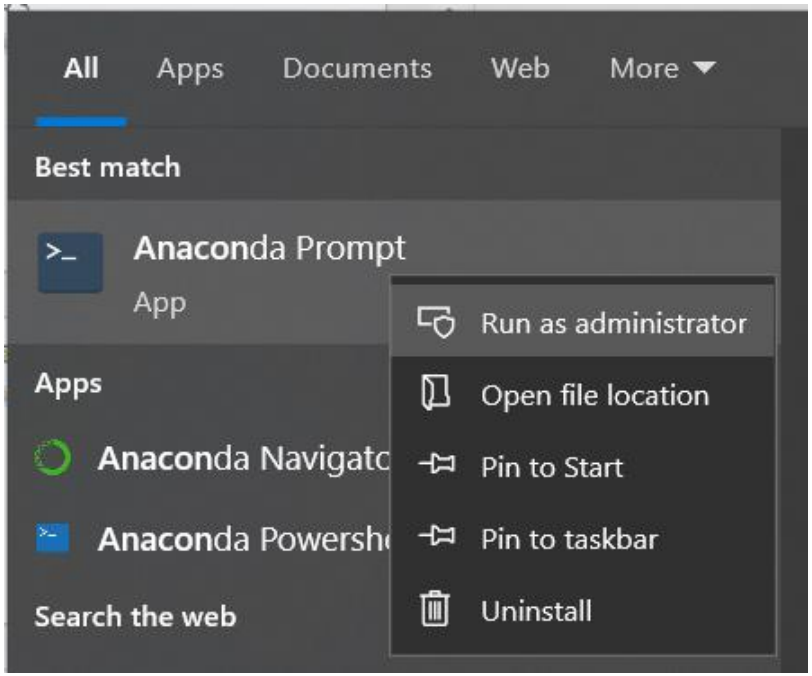

Data (D:) › ITCS227-lab01
Search ITCS227-lab01
Name
Date modified
Type
Size
ITCS227_Lab01_Tutorial.ipynb
1/2/2025 11:30 PM
IPYNB File
20 KB
Copy and paste the ipynb file.


jupyter
Quit
Logout
Files
Running
Clusters
Select items to perform actions on them.
Upload
New
0
/
Name
Last Modified
File size
ITCS227_Lab01_Tutorial.ipynb
13 hours ago
20.1 kB
Refresh jupyter and click on the file.


jupyter ITCS227_Lab01_Tutorial (autosaved)
Logout
File
Edit
View
Insert
Cell
Kernel
Widgets
Help
Kernel starting, please wait...
Not Trusted
Python 3 (ipykernel)
Run
Markdown
Mahidol University
Faculty of Information
and Communication Technology
Tutorial 01: Using Jupyter Notebooks
Jupyter Notebooks combine executable code with formatted text, images, and other content within cells. These formatted cells use Markdown and are key for documenting and explaining the notebook's contents. This lab will focus on practicing Markdown writing.

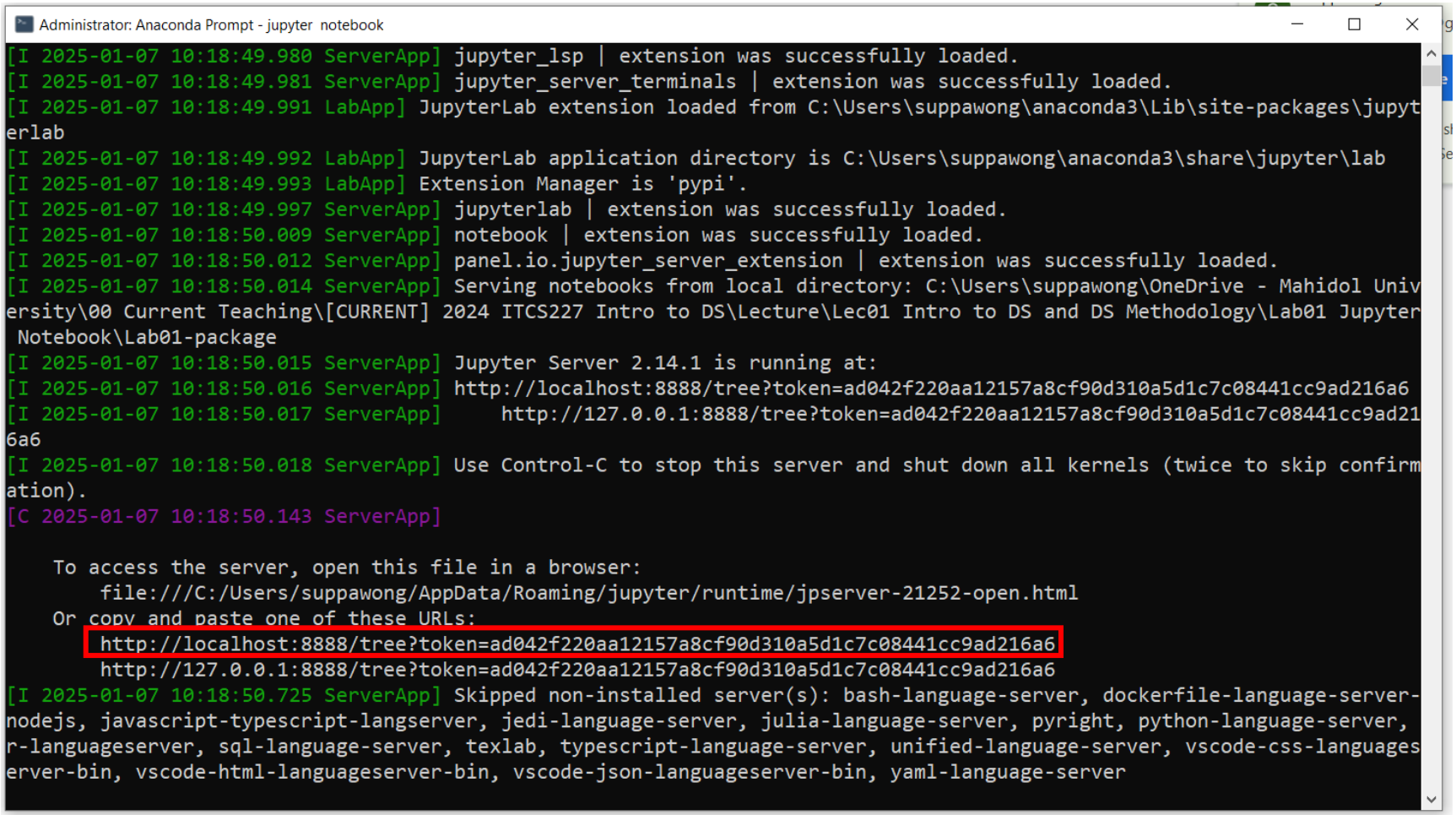
# If you have “Permission” issues.

- Run Anaconda Prompt as administrator.
- Follow the previous steps.



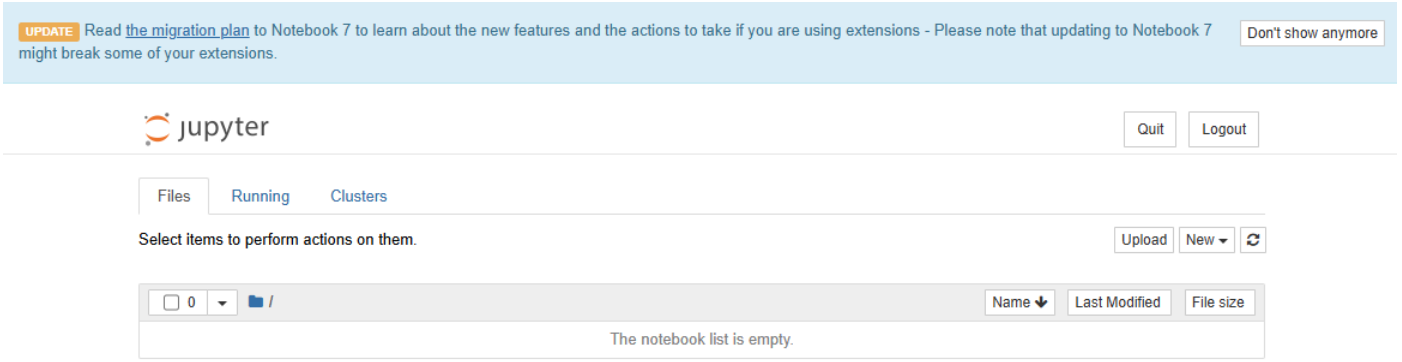
# If you have “Permission” issues (cont)

Copy and paste the URL from the prompt.



# The Jupyter Dashboard

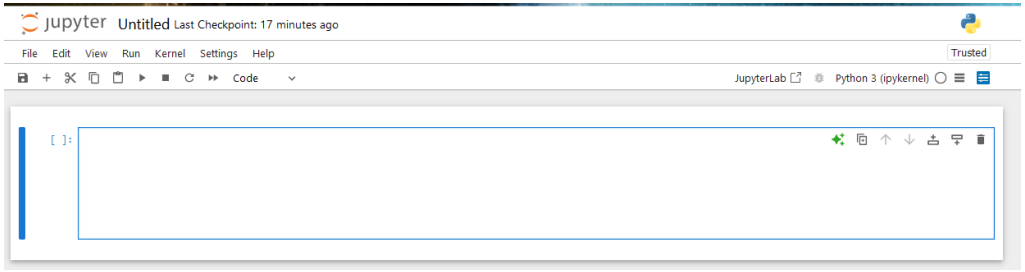
- After launching, you'll see the Jupyter Dashboard.
- This is your file browser.
- You can navigate through your folders here.
- To create a new notebook, click the "**New**" button on the top right and select "**Python 3 (ipykernel)**".



# Command Mode vs. Edit Mode

Jupyter has two modes for interacting with cells:

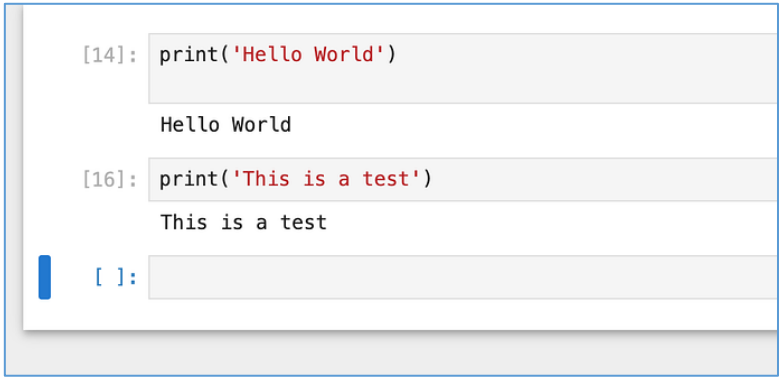
- **Command Mode (Blue border)**: You are interacting with the cell as a whole. Press `Esc` to enter. Used for: moving, copying, deleting, and changing cell type.
- **Edit Mode (Green border)**: You are typing inside the cell. Press `Enter` to enter. Used for: writing code or text.

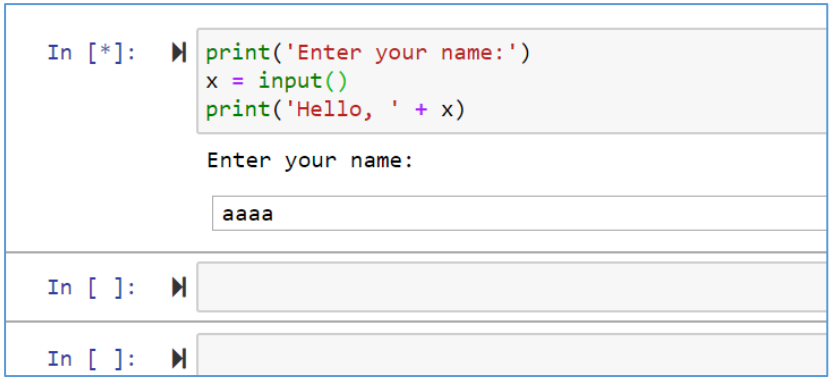





# Running a Cell

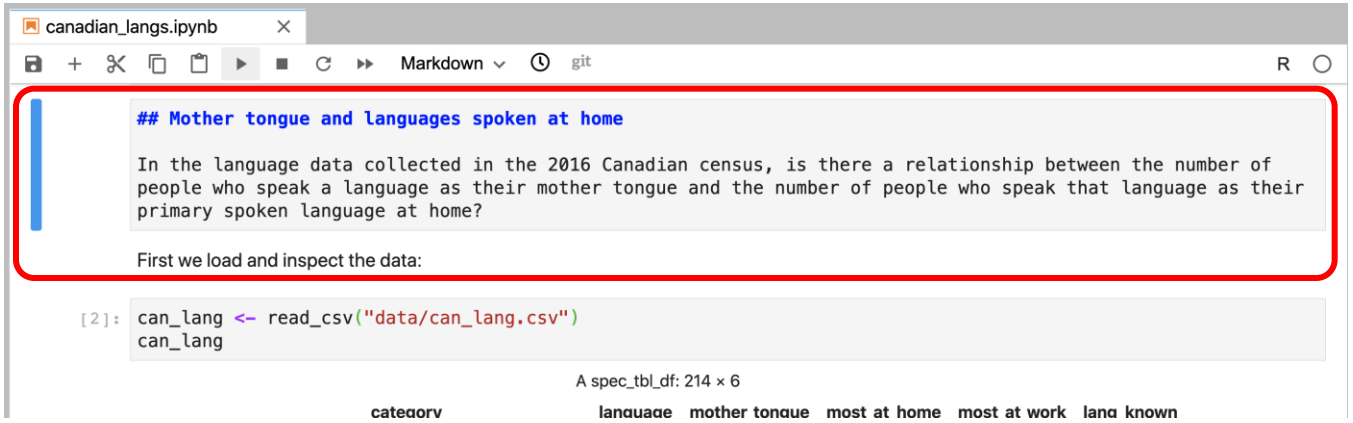
- To run the code or render the text in a selected cell, press:
  - **Shift + Enter**: Run the cell and select the cell below. (This is the most common way.)
  - **Ctrl + Enter**: Run the cell and keep the current cell selected.
- When a code cell is running, an asterisk [*] appears next to it. When it's done, it's replaced by a number [1].



# Cell Type 1: Markdown Cells

- Used for all your text: headings, explanations, lists, and links.
- It uses a simple formatting language called **Markdown**.
- To use it:
  - Select a cell.
  - In the toolbar, change the cell type from "Code" to "Markdown".
  - Type your formatted text.
  - Run the cell (`Shift + Enter`) to see the beautifully rendered result.





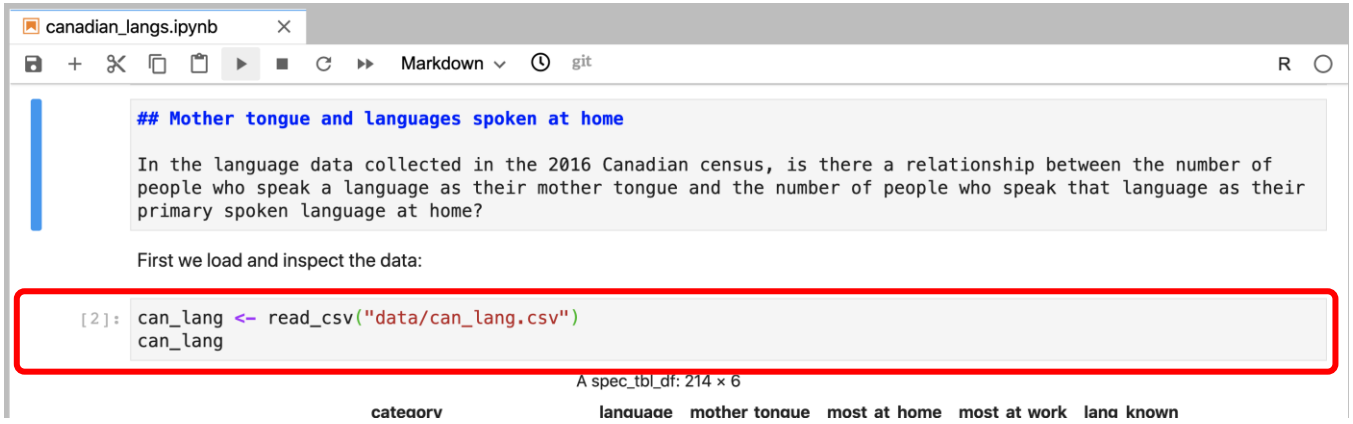
# Basic Markdown Syntax

Here are the key formats you'll need for your lab reports:

- Headings: Use hashes (#). More hashes = smaller heading.
  - `# Level 1 Heading`
  - `## Level 2 Heading`
- Emphasis:
  - `*italic text*` -> *italic text*
  - `**bold text**` -> **bold text**
- Lists:
  - Start a line with `*` or – for a bulleted list.

# Cell Type 2: Code Cells

- This is where you write and execute Python code.
- It's the default cell type.
- Any output from your code (like a calculation result or a plot) will appear directly below the cell after you run it.





# The Notebook Toolbar

- The toolbar gives you quick access to common actions.
- Key buttons to know:
  - **Save**: (Floppy disk icon) Save your work!
  - **Add Cell**: (+) Inserts a new cell below the selected one.
  - **Cut/Copy/Paste**: (Scissors/Files icons) Manipulate cells.
  - **Run**: (Play button) Executes the current cell.
  - **Interrupt**: (Stop button) Stops a long-running calculation.



# Getting Help

- **Kernel Menu**: If your code gets stuck or things seem weird, try restarting the "kernel" (the engine that runs your code).
- Go to `Kernel -> Restart` or `Kernel -> Restart & Clear Output`.
- **Help Menu**: Contains useful links to documentation for Python and libraries.



# Basic Python Programming



# Storing Information: Variables

- A **variable** is a name that refers to a value. It's like a labeled box where you can store information to use later.
- To create a variable, you use the assignment operator =.

```
variable_name = value
```

- Variable names cannot have spaces and should be descriptive.



# Using Variables in Calculations

- Once you store a value in a variable, you can use the variable's name in your calculations just as you would use the number itself.
- This makes your code much more readable and maintainable.

Bad:

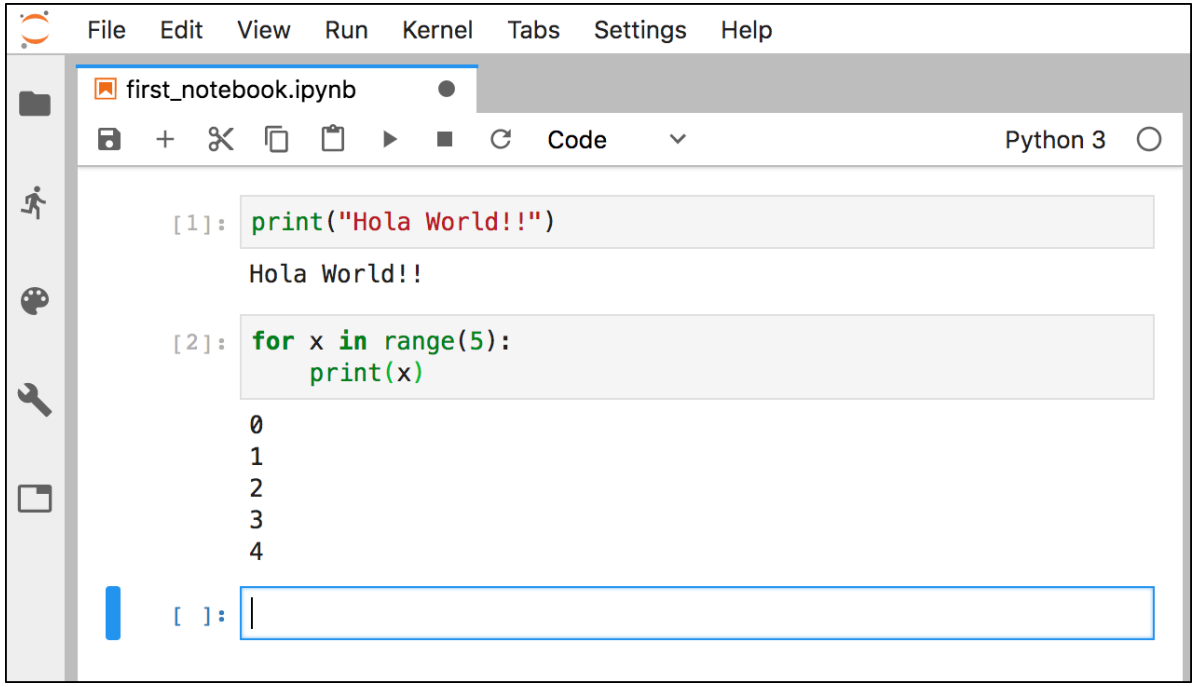
```
area = 12.5 * 10.0
```

Good:

```
length = 12.5
width = 10.0
area = length * width
…
volume = length * width * height
```

# Displaying Output: The `print()` Function

- A code cell only shows the output of the very last line.
- To see multiple outputs or display text and variables together, use the `print()` function.
- `print()` is a fundamental tool for inspecting your code's behavior.



# Formatting Output with f-strings

- "f-strings" are the modern, easy way to mix text and variables inside a `print()` statement.
- Just put an `f` before the opening quote and place your variable names inside curly braces `{}`.
- Syntax: `print(f"The value of my_variable is {my_variable}")`

```
In [1]: city = "Bangkok"
        temp = 35
        print(f"The current temperature in {city} is {temp} degrees.")

        The current temperature in Bangkok is 35 degrees.
```



# Data Types: Numbers vs. Text

- Python treats different types of data differently.
- **Numbers: (int for integers, float for decimals) are for math.**
  - `age = 20`
  - `price = 99.50`
- **Strings**: (str for text) must be enclosed in quotes (" or ‘).
  - `student_name = “Somsak”`
- You can't do math with strings: `5 + "Somsak"` will cause an error!

# Leaving Notes: Comments

- Code explains **how**, comments explain **why**.
- In Python, any text on a line after a hash symbol (#) is a **comment**.
- The computer ignores comments completely. They are for humans!
- Use them to explain complex parts of your code to your future self and others.

```
In [2]: # Calculate the area of a circle
        radius = 5
        area = 3.14159 * radius**2 # Using the formula A = pi * r^2
        print(area)

        78.53975
```